

Hands-on Exercise No. 3 DigiSkills
3.0 Batch-04
Full Stack Development with MERN

Total Marks: 10
Due Date: 01/10/2026

Instructions:

Please read the following instructions carefully before submitting this Hands-on Exercise:

- Use MS Word to prepare exercise solutions.
- You may consult tutorials and videos if the concept is not clear.
- Your submitted exercise will not be considered/counted if:
 - It is submitted after due date.
 - It is not in the required format (doc or .docx)
 - It does not open, or file is corrupt.
 - It is copied (partial or full) from any source (websites, forums, students, etc.)

Learning Outcome:

After completing this exercise, you shall be able to:

- Create reusable React components for a small Blog Dashboard interface.
- Apply React hooks such as useRef, useMemo, useCallback, and custom hooks in a practical frontend task.
- Use React.memo to reduce unnecessary component re-rendering in a React UI.
- Configure Redux Toolkit store and slices for managing global application state.
- Use useSelector and useDispatch to read and update Redux state inside React components.
- Prepare a proper solution file by adding screenshots of code, browser output, and terminal output.

Problem Statement

In this hands-on exercise, you are required to build a small React Blog Dashboard. You will apply React hooks, custom hooks, memorization, component reuse, and Redux Toolkit state management. The objective is to strengthen your React frontend development skills and understand how state can be managed in a scalable React application.

You can use VS Code and a React Vite project to complete this exercise. You may use normal CSS or Tailwind CSS for frontend styling.

After completing each task, take screenshots of your code and its output, and paste them into a single MS Word (.doc/.docx) file as your final solution submission.

Tasks:

Question 1: React Blog UI with Hooks (Marks 5)

Build a React Blog Dashboard using reusable components and React hooks.

Use the following sample blog data in your project:

```
const initialBlogs = [
  {
    id: 1,
    title: "Understanding React Hooks",
    category: "React",
    author: "Ali Khan",
    readingTime: 6,
    featured: true
  },
  {
    id: 2,
    title: "Redux Toolkit Basics",
    category: "Redux",
    author: "Sara Ahmed",
    readingTime: 8,
    featured: false
  },
  {
    id: 3,
    title: "Building Blog UI in React",
    category: "React",
    author: "Hamza Malik",
    readingTime: 5,
    featured: false
  },
  {
    id: 4,
    title: "Node.js and Express Introduction",
    category: "Node",
    author: "Ayesha Noor",
    readingTime: 7,
    featured: true
  }
];
```

- Part 1: Create a React Vite project and build the following components: Header, SearchBar, BlogCard, BlogList, and BlogStats. (Marks 1)
- Part 2: Display all blogs using an array and map() method. Each BlogCard should show title, category, author, reading time, and featured status. (Marks 1)

- Part 3: Create a custom hook named useFilteredBlogs. Use useMemo inside this hook to filter blogs by search text and category. (Marks 1.5)
- Part 4: Use useRef to focus on the search input when the page loads. Use useCallback for delete/favorite button functions and React.memo for BlogCard. (Marks 1)
- Part 5: Apply CSS or Tailwind CSS to make the dashboard clean, responsive, and readable. (Marks 0.5)

Question 2: Redux State Management for Blog Dashboard (Marks 5)

Add Redux Toolkit state management to the Blog Dashboard.

- Part 1: Install Redux Toolkit and React Redux. Create store.js and blogSlice.js files. (Marks 1)
- Part 2: Store blogs, search text, selected category, loading, and error state inside Redux. (Marks 1)
- Part 3: Create reducers/actions for add blog, delete blog, toggle featured, set search text, and set category. (Marks 1.5)
- Part 4: Use useSelector and useDispatch in React components to read and update Redux state. (Marks 1)
- Part 5: Add one async example using createAsyncThunk or a simulated API loading state. Show loading and error message handling in UI. (Marks 0.5)

Marking Scheme:

Question / Part	Requirement	Marks
Question 1 - Part 1	React Vite project and required components	1
Question 1 - Part 2	Blog list rendered using array and map()	1
Question 1 - Part 3	Custom hook and useMemo filtering	1.5
Question 1 - Part 4	useRef, useCallback, and React.memo usage	1
Question 1 - Part 5	Responsive UI styling with CSS or TailwindCSS	0.5
Question 2 - Part 1	Redux Toolkit setup with store and slice	1
Question 2 - Part 2	Required Redux state values added	1
Question 2 - Part 3	Reducers/actions for blog operations	1.5
Question 2 - Part 4	useSelector and useDispatch integration	1
Question 2 - Part 5	Async loading/error state example	0.5

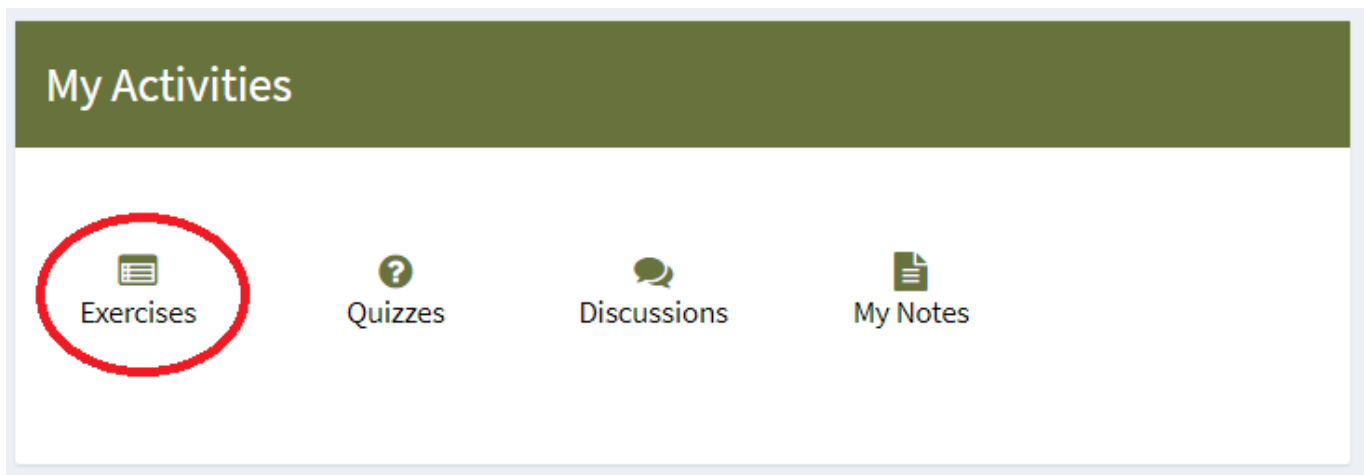
NOTE: Save the code files at your end and only provide screenshots of each step in MS Word File

BEST OF LUCK 😊

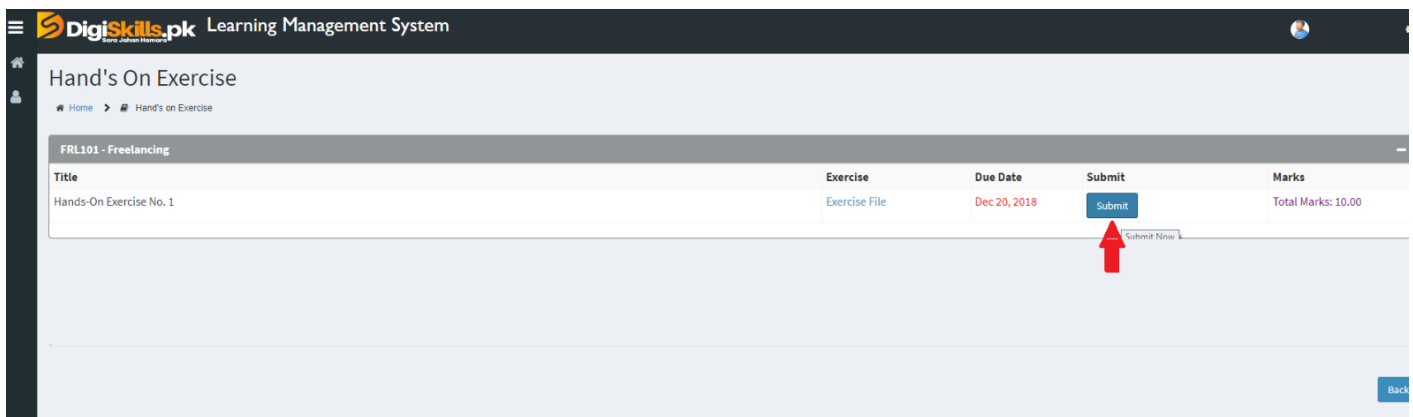
How to submit solution file on LMS?

Please take the following steps for submitting your solution using LMS:

- 1) Login to the LMS
- 2) Click on the **Exercises** button within the **My Activities** section



- 3) Click on the submit button to upload your Solution.



- 4) Keep in mind to upload your Solution in .doc or .docx format